Table of Contents

Part 01 – Environment Setup 2

Part 02 – Pushing to GitHub 5

Part 03 – Pulling from GitHub 6

Part 04 – Pull Request 10

Part 05 – Updating Local after Pull Request 13

Part 06 – Changes on the Remote 14

Part 07 – Forking a Repo on the Remote 18

Part 08 – Branch on GitHub.com 22

Part 09 – Creating an Issue on GitHub 26

Appendix A – Tracking a Fresh Directory 30

Appendix B – Fresh Token Generation 30

Appendix C – Show .git Folder in VS Code 31

Git is a version control system that makes it easy to just "undo" a change. It is like a time machine for code. Instead of saving multiple files like file01.js or file02.js, Git tracks every small change made to a single project folder. This allows developers to experiment with new features safely. For a budding developer, it means that you can take risks, have a clear history of your learning progress, and collaborate with other engineers on shared codebases.

In the first part (Git Bootcamp) we installed Git locally on an Ubuntu OS. In this bootcamp we will extend the knowledge of Git to GitHub. GitHub is an online repository, a place to store and share snapshots of Git directories. So, Git locally is the tool but GitHub remotely is the storage unit.

Before you begin this bootcamp, get an account at GitHub, it is free and easy to setup.

If you are continuing today from where we left off yesterday we will be starting over. You will receive a zipped file named git-node-d2-starter.zip. I have removed all traces of Git from this directory. We will begin as though we never had Git setup on this project. You will also receive a .gitignore file or you can create one on your own. I also deleted the remote repo from GitHub.com.

Note, if you participated in the Git locally bootcamp and pushed your project to GitHub.com, you should remove and delete everything from the remote repo and on your local computer as well. What we want is to start over. The files you downloaded for today has a zipped file called git-node-d2-starter.zip. This is the file you want to start with; it already has a project for you.

Part 01 – Environment Setup

1. Unzip the file you downloaded. It is the same project from day 1 but I removed all traces of Git. If you want, you can rename the folder to *git-node-d2* so that we can distinguish today's work from yesterday. You can check if any directory is Git-controlled by running this command:
   `git rev-parse --is-inside-work-tree`
   If it returns "true", it is a Git repository.
   Open the new folder in VS Code and open a new terminal in VS Code. Lets now initialize Git like we did yesterday:

   ```
   git init
   ```

2. Just to be sure run the following commands again to make sure we have these *globals* in place:

   ```
   axle@pc0480:~$ git config --global user.name "AxleB"
   axle@pc0480:~$ git config --global user.email
   "axle.barr@skillsoft.com"
   ```

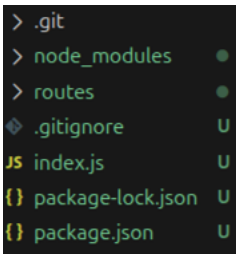
   The highlighted values will be your own values, please do not use these exact values.

3. If you did not setup VS Code as your default editor on Day01, then you can easily do it now with this simple Git command:

```
git config --global core.editor "code --wait"
```

Use `git config --global --list` to check for existing global settings.

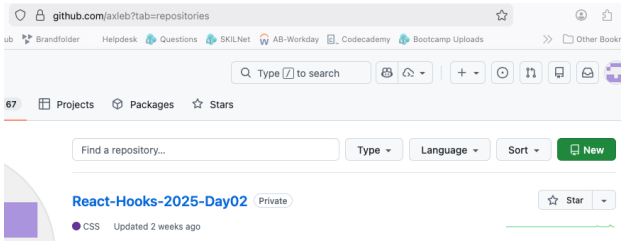
4. Add the .ignore file back into the folder for today, if not there. Run the `npm install` command. At this point your folder structure should look like this in VS Code:



Note: If you use the zipped file provided, you will already have the git ignore file.

5. We can now make the connection between the Git on our local machine and GitHub.com. Start from the GitHub end, log in and begin by creating an empty repository. Remember you should have deleted the remote repository from yesterday, if you did not, the new repo will have a new name. It will have the *d2* suffix. There are several ways to work with Git and GitHub, this is just one of them.

Once you are on github.com, go to the *repositories* tab and on the top right there should be a button labeled as **New**, click it to begin the new repository process.

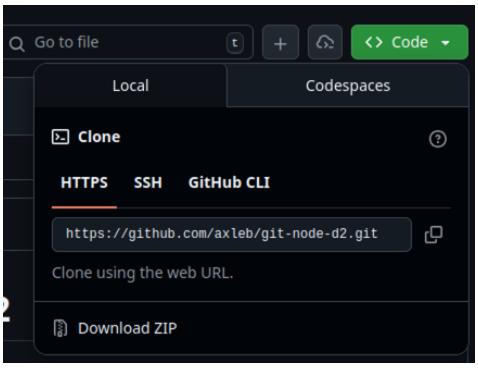


Repository name: Give it a name (e.g., **git-node-d2**).
Optionally you can give it a description and change *owner* to you, if you are part of an organization.

Settings: Keep it Public and for today do not yet initialize it with a README file. Also do not choose a license, or .ignore file (since you already have these locally). If you have access to multiple repositories, you will need to choose the appropriate one via the "Owner" selection box.

Click Create repository

6. To get the remote location URL, you will need to click the drop-down arrow on the green Code button. Clicking that button will reveal several options and the *Clone-Https* option is the one you want for remote access. Copy this address to your clipboard or store it in a plain text file for later use. There is a copy icon to the right of that box:



If you just created a new repository then that first page will have all the information you need.

7. Back to your local computer (or VM/Sandbox) complete the next command but change *axleb* to YOUR user account name on GitHub.com:

```
git remote add origin https:github.com/axleb/my-node-app.git
```

This is the username you chose when you created an account on GitHub.com. It is also available on GitHub.com, once the second HTML interface page opens.

If this command is successful you can always double check using this command:

```
git remote -v
```

Part 07 #10 will show a different command to connect the two repositories.

Part 02 – Pushing to GitHub

Once we have our local and remote repos setup, it is time to reconcile and try to get both local and remote in sync with each other. We will start by organizing the local repo first.

1. If you do a `git status` on the local directory you will see almost all the files are red. This is because we started today with a new folder and added Git afterwards. On Day 1 we began with a Git initialized folder. Nevertheless, the process is the same first add all files to Git:

   ```
   git add .
   ```

   Remember to add the .gitignore file before you run this command. You will perform a git commit in the next part.

1. Do your first commit locally, for this project today:

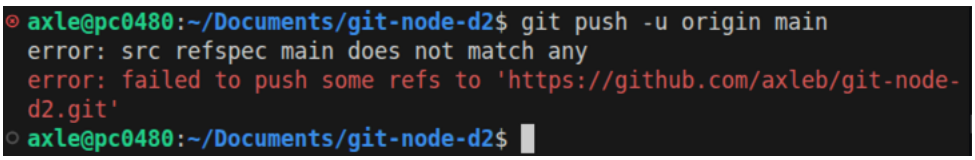
   ```
   git commit -m "First commit for day2"
   ```

2. Now we can do the first `push` for today:

   ```
   git push -u origin main
   ```

   A push will try to get your local files to the remote repo. **Prepare for several errors/warnings/obstacles.** The **-u** simply guides the data upstream, it saves from typing "origin main" for every future command.

3. If you get an error like this, it probably means that there is a name conflict. Yesterday we changed *master* to *main*, see #4:

4. On Day01, Part10 #5 we changed the name of master to main. Today we will use a CLI command:

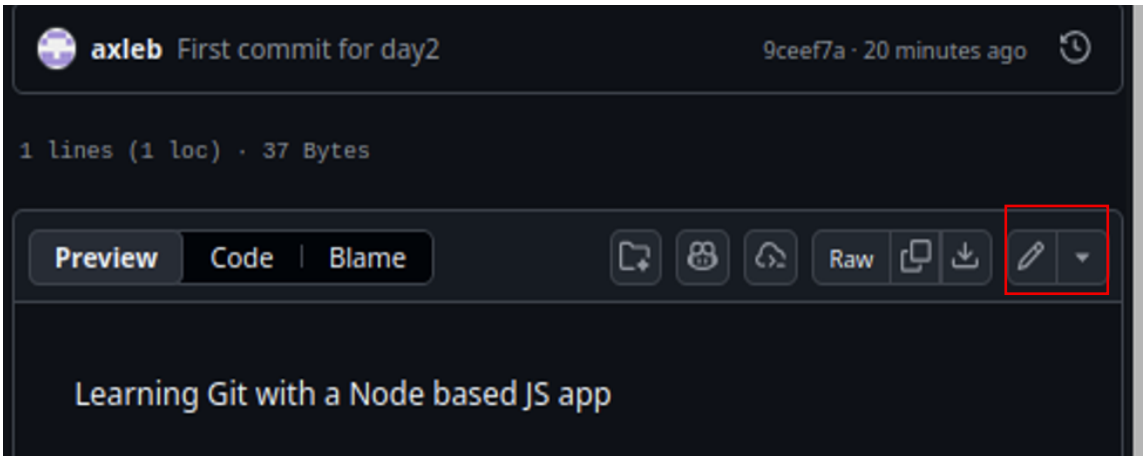
   ```
   git branch -m master main
   ```

   Confirm with a `git status` check, or check the VS Code IDE, bottom left.

5. At this point your remote and local should be the same except for the node_modules folder. Remember we are not supposed to track this folder due to its size.

Part 03 – Pulling from GitHub

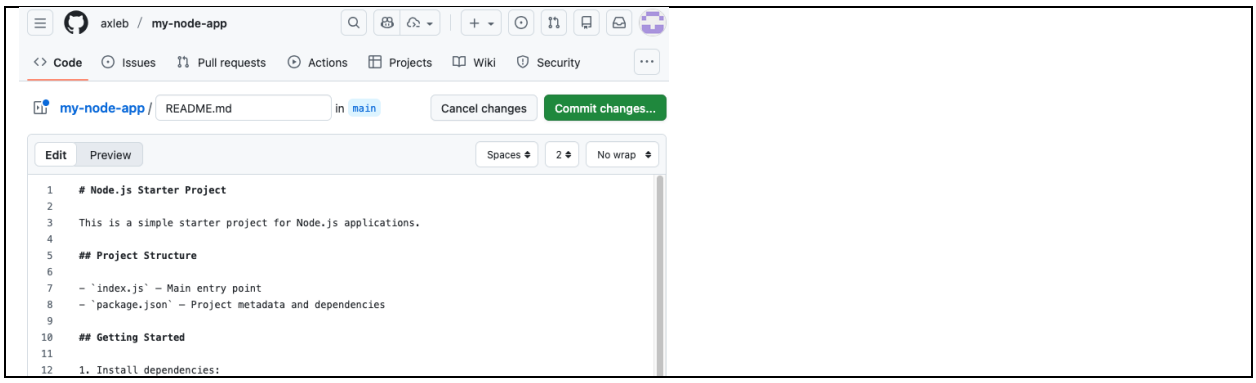
We will now edit the README.md file on the remote repository by describing our starter project. On the remote repo, you can start by clicking on the file you want to change, in this case it is the README file. You will be taken into a separate UI where you can click on the pencil looking icon to start editing the file right there on the web:



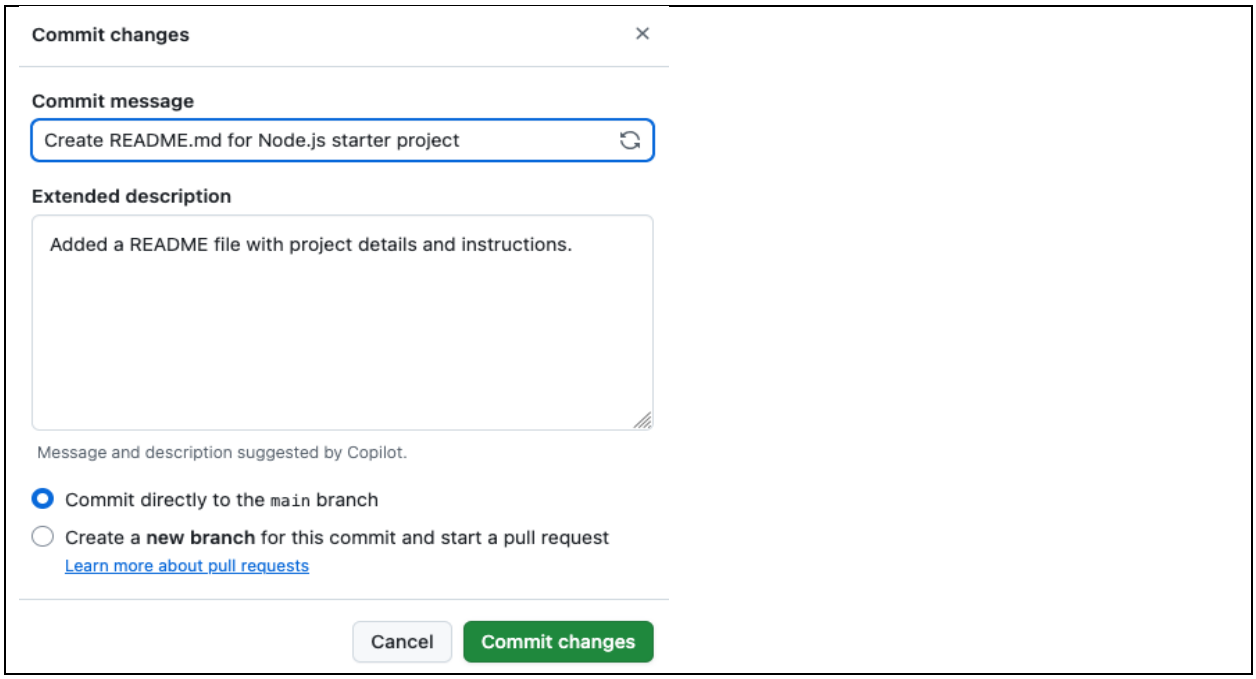
1. You can copy as much of the following text as you wish for the readme file. Remember this file is being updated on the GitHub.com website, not locally:

```
# Node.js Starter Project
This is a simple starter project for Node.js applications.
## Project Structure
- `index.js` – Main entry point
- `package.json` – Project metadata and dependencies
## Getting Started
1. Install dependencies:
   ```bash
   npm install
   ```
2. Run the application:
   ```bash
   node index.js
   ```
Requirements
- Node.js (v14 or higher recommended)
- npm (Node Package Manager)
License
MIT
```

2. Once you have added text to the README, just click the `Commit changes...` green button, at the top right corner:



3. Add an appropriate message and click the green `Commit` changes button on the dialog box that shows up:



Note, we are committing directly to the `main` branch on the remote repo.

4. After these changes, the remote repository will be out of sync with the project on our local computer. The command on the local machine to get the changes is very simple:

```
git fetch origin
```

This updates your local copy of the remote-tracking branches (e.g., `origin/main`), which is crucial for an accurate comparison

```
axle@pc0484:~/Documents/git-node-d2$ git fetch origin
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 1.23 KiB | 1.23 MiB/s, done.
From https://github.com/axleb/git-node-d2
   9ceef7a..7144eda  main       -> origin/main
```

5. Now do a git status and it would report back something like you see below:

```
axle@pc0480:~/Documents/git-node-d2$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast
rded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean_
```

Git is reporting that the local branch is behind by 1 commit. It is also telling us how to proceed. If your status show "nothing to commit, working tree clean" then try this command:
`git log main..origin/main –oneline`
If it shows something like this below, then do a `git merge origin/main`

```
axle@pc0480:~/Documents/git-node-d2$ git status
On branch main
nothing to commit, working tree clean
axle@pc0480:~/Documents/git-node-d2$ git log main..origin/main --oneline
52b98a2 (origin/main, origin/HEAD) Revise README for Node.js starter projec
t
```

6. Run the pull command to bring the local branch up to date with remote:

```
git pull
```

After the command open the README.md file and you will see all the text we added on the remote file. If that did not work, try a `git fetch origin.` Then do a `git reset --hard origin/main.`

```
axle@pc0484:~/Documents/git-node-d2$ git pull
Updating 9ceef7a..7144eda
Fast-forward
 README.md | 20 ++++++++++++++++++-
 1 file changed, 19 insertions(+), 1 deletion(-)
```

At this point both repositories are up to date and synchronized. If you wanted to check at any time if both are in sync run the commands `git fetch origin` and `git status`. The git status should tell you that the working branch is up to date with the remote branch.

6. Lets now make a change to the local README and then push that one file to the remote. In the local README lets change the two requirements like below:

```
   git rm readme.md
   ## Requirements
       - Node.js (v20 or higher recommended)
       - npm (Node Package Manager v11 or higher)
       ## License
MIT
```

So just really minor changes but it helps us understand the push and pull a bit more.

7. If you did a git status, you will notice that the README has changed. Now, since we are only working with just the one file, let's just add that one to the staging area:

```
git add README.md
```

Since the README file is at the root, there is no path as such, but you may need an actual path if your file is deeper in the project file system.

8. Now for the commit:

```
git commit -m "Updated the versions for Node and NPM in README"
```

9. Finally lets do the push to our remote repo:

```
git push origin main
```

Now if you refresh your GitHub page which should be pointing to the remote README.md file, you should see the changes there:



Part 04 – Pull Request

We will create a new branch on the local repo, add a new feature, then push that branch to the remote repository. In the Git bootcamp, Day01, we did create a new branch and merged the changes back into the main branch, but we did not update any remote repositories.

We will also create a Pull Request (PR). This is when we are "requesting" that the owner of the project "pull" changes from an experimental branch into the main one. In this case I am the owner of the project.

1. Let's first create a new branch locally if we want to create a PR, so in the terminal create a new branch:

```
git checkout -b controller-feature
```

Note, a PR can happen between any two branches, it does not have to involve main. After the above command, the terminal window should update with a new message.

2. Create a new directory called controllers and create a new `.js` file called controller.js

3. Open the controller.js file in an editor and start entering the first controller function. Controllers will take responsibility for making decisions. The controller should handle what happens when the user navigates to the root route:

```
exports.getdefault = function(req, res){
   res.send('You are on the root route.');
};
```

In this case we are not exporting the entire file, but each function is exported individually.

4. Continue to develop this file by completing all the route functions, in other words, write functions that match the routes we had before. For now these functions are very simple, but they could become a bit more complicated:

```
exports.getdefault = function(req, res){
  res.send('You are on the root route.');
};
exports.aboutus=function(req, res){
  res.send('You are on the about us route.');
};
//
exports.addemployee=function(req, res){
  res.send('You are on the addemployee route');
};
```

5. Back in the routes.js file, we need to let this file know that there is a controller handling each route, so basically routes.js is now acting like a pointer to a controller function, which does the final piece in deciding what to serve to the client. Add this line at the top of the function in routes.js.

```
const controller = require('../controllers/controller');
module.exports = function(router){
  router.get('/', (request, response)=> response.send('Hello from Skillsoft!'));
  router.post('/addemployee', (request, response) => {
```

6. Remove everything from inside of the parenthesis of each function and just use a pointer to the controller file:

```
const controller = require('../controllers/controller');
module.exports = function(router){
  router.get('/', controller.getdefault);
  router.get('/aboutus', controller.aboutus);
  router.post('/addemployee', controller.addemployee);
};
```

7. So three pointers to three functions in the controller file. Remember the /addemployee endpoint is a POST request not a GET request like the others.

8. Run the Node application to make sure that this new controller feature works properly and no errors are thrown. The NPM command `npm run start` will start up the Node application.

9. Lets normalize the branch:

```
axle@pc0480:~/Documents/git-node-d2$ git add .
axle@pc0480:~/Documents/git-node-d2$ git commit -m "Added a controller featur
e"
[controller-feature 7da8894] Added a controller feature
 2 files changed, 15 insertions(+), 6 deletions(-)
 create mode 100644 controllers/controller.js
axle@pc0480:~/Documents/git-node-d2$ git status
On branch controller-feature
nothing to commit, working tree clean
axle@pc0480:~/Documents/git-node-d2$
```

Now we can do the merge

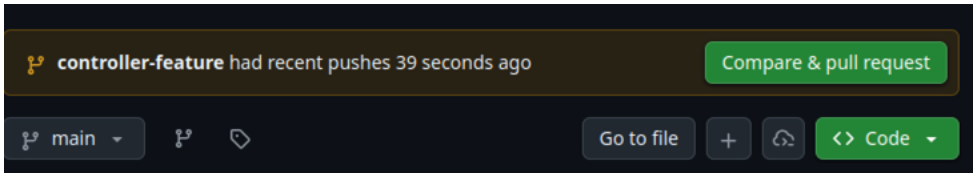
10. (Optional) In my case, I realized that I made an error with the spelling of the name of the feature branch. This is how you would correct a mistake like that:

```
axle@pc0484:~/Documents/git-node-d2$ git status
On branch conroller-feature
nothing to commit, working tree clean
axle@pc0484:~/Documents/git-node-d2$ git branch -m controller-feature
axle@pc0484:~/Documents/git-node-d2$ git status
On branch controller-feature
nothing to commit, working tree clean
```

11. Make sure that you are on the feature branch then push the branch itself:

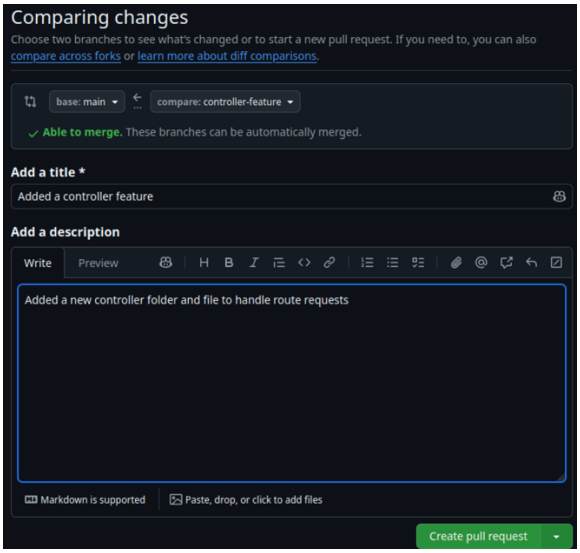
```
git push origin -u controller-feature
```

You may have to go through the sign in process again



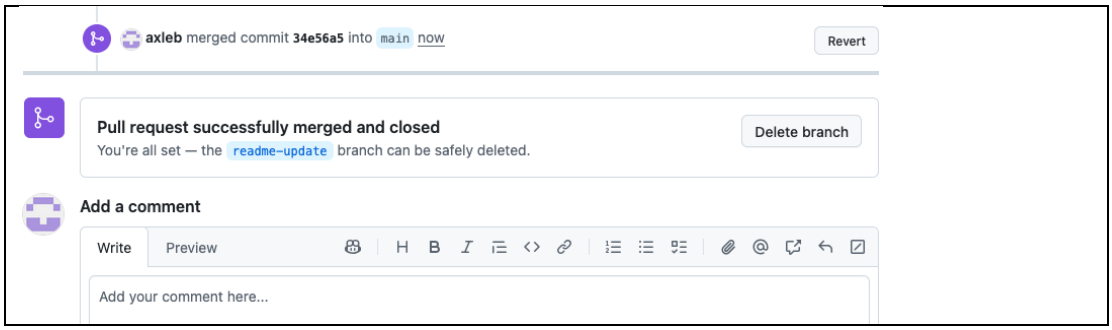
If you refresh the remote repository you will see a message at the top. Notice the **compare & pull request** green button.

12. Next to the banner above, to the right, click on the green `Compare and pull request`:



You can give it a description and click on the `Create pull request` green button at the bottom right of that page.

13. At this step, you should see a green `Merge pull request` at the bottom of the same page. Also there should not be any conflicts. Click the green button and then Confirm merge:



Also you will need to *confirm* the merge.

Part 05 – Updating Local after Pull Request

If you did a status check on the controller-feature branch on the local machine, it will show nothing to commit and a clean working tree. Go back to the main branch and do a status there. You can use the command `git switch main`.
Once you go back to the main branch do a git status. In my case it reported that all is well but this is incorrect. We will now update main via the remote repo.

1. Let's first try to update the local main branch:

```
git pull origin main
```

This assumes that you are already on the main branch locally. If not, you can do a `git checkout main`, it's the same as switch.

```
axle@pc0484:~/Documents/git-node-d2$ git pull origin main
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 900 bytes | 900.00 KiB/s, done.
From https://github.com/axleb/git-node-d2
 * branch            main       -> FETCH_HEAD
   ab212b9..28e2055  main       -> origin/main
Updating ab212b9..28e2055
Fast-forward
 controllers/controller.js | 10 ++++++++++
 index.js                  |  8 ++------
 routes/routes.js          | 12 +++++-------
 3 files changed, 17 insertions(+), 13 deletions(-)
 create mode 100644 controllers/controller.js
```

2. If you forgot that you had a second branch this command will reveal all branches:

```
git branch
```

3. Now we should delete the feature branch:

```
git branch -d controller-feature
```

4. Follow up with this command which will inform your local machine that the remote branch no longer exists and that it should not keep any memory of it:

```
git fetch --prune
```

5. Here is a slightly different sync check you can use. You can check if there is something in the local that needs to be pushed (or vice versa):

```
git log origin/main..main
git log main..origin/main
```

6. In my case both commands returned nothing, so all repos are up to date:

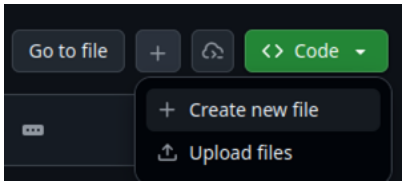
```
axle@pc0484:~/Documents/git-node-d2$ git log origin/main..main
axle@pc0484:~/Documents/git-node-d2$ git log main..origin/main
axle@pc0484:~/Documents/git-node-d2$
```

Part 06 – Changes on the Remote

At this point, you have local and remote repositories and they are synchronized. On the remote we still have the controller-feature branch but it is not part of our discussion for now.

We will now make a major change on the remote and then reconcile with the local project. We will attempt to add database support to our app but on the remote repository. Follow the steps carefully as it is easy to make a misstep here.

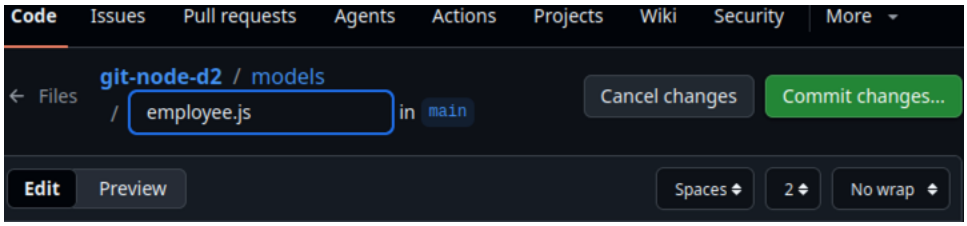
1. On the remote repository on GitHub.com, on the main branch, click on the **+** icon, between the *Go to file* and *cloud* icon. This will allow you to add a directory:



Click on the `Create new file` option. *Note if your browser is stretched, you will NOT see the **+** icon, you will see "Add file" instead*.

2. Click in the box just under the name of the repo. It will say *Name your file…* but if you just enter *models* followed by a / it will create a folder first. So type in that box:
`models/`

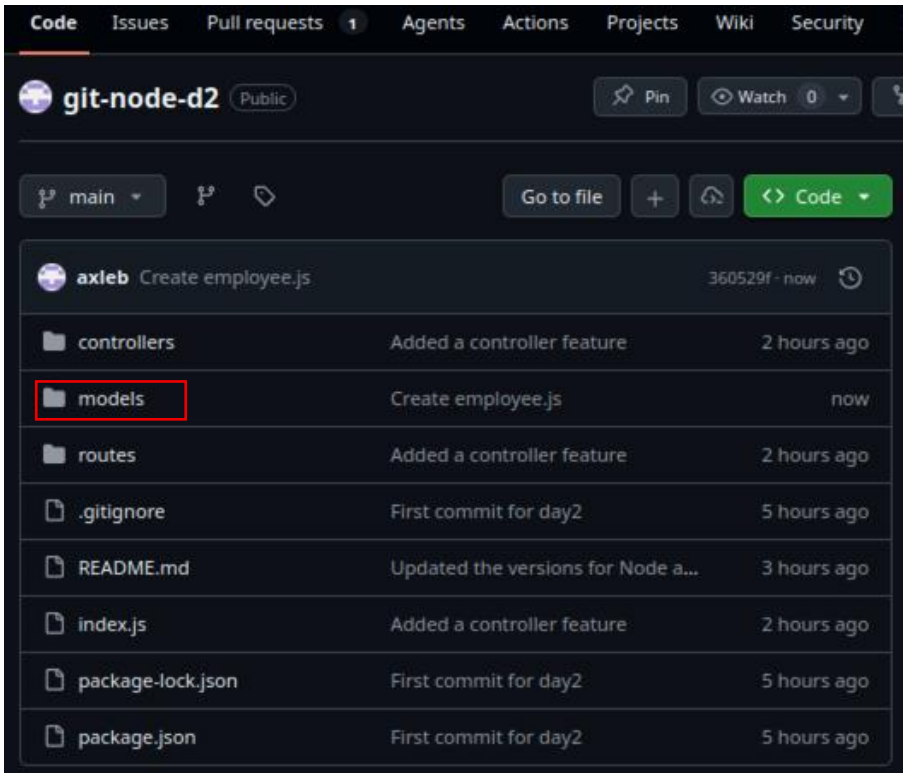
3. Now it says *Name your file* again, so type into the box the file name of employee.js



Click on the *Commit changes*…

4. Click on the *Commit change*s on the new dialog box that pops up. You could add a description if you want.

5. Your interface should now look like the image below.
   So we have a new folder called models and a new file called employee.js. It is empty for now:



6. In your downloaded files for today you have a file called employee.js, you can either upload and replace the file you just created or copy the contents of the downloaded file and paste it into the remote file.



   Once you choose an option just make sure the remote employee.js has the same code as the downloaded employee.js. See the image below:

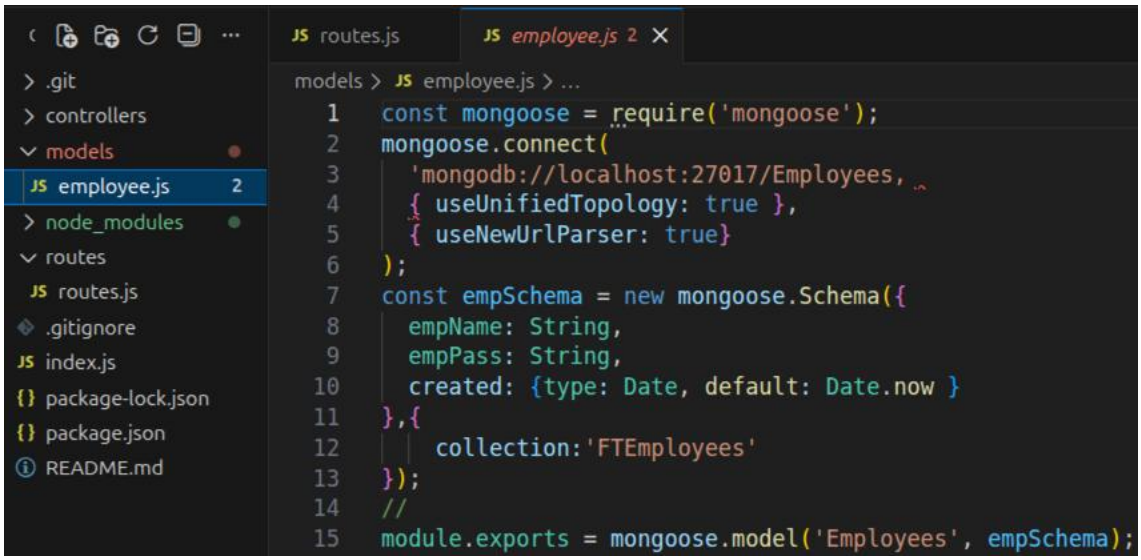
7. Once again our two repos are out of sync. Back on the local machine run a
   `git fetch origin`
   After running this, your local Git knows about the changes, but your files won't change until you merge them.

8. If you want to see the changes in the terminal, run `git diff main origin/main`

```
axle@pc0480:~/Documents/FSD-D1-P07$ git diff main origin/main
index 4228cd8..5ff49f2 100644
--- a/index.js
+++ b/index.js
@@ -10,5 +10,5 @@ app.use(express.urlencoded({extended:false}));
 app.use(router);
 //
 app.listen(port, function(){
-       console.log("Listening " + port);
+       console.log("Listening on " + port);
:...skipping...
diff --git a/index.js b/index.js
index 4228cd8..5ff49f2 100644
--- a/index.js
```

The image above is from a different procedure. Remember the *spacebar* will continue showing content and typing the letter *q* will exit the display.

9. Now for the update, here is the command in the terminal window: `git pull origin main`
You can verify the change is on the local machine just by looking at the VS Code file explorer panel and checking the content of the employee.js file:

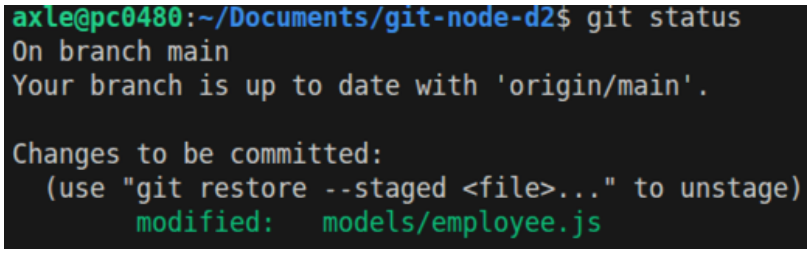


Notice that the models directory is reddish. The project is up to date but the code is showing a syntax error. Just add the single quote after `Employees` on line 3. This will make both projects out of sync again. By now you should have the commands mostly memorized, see below:

10. In order to sync local and remote run the following commands in sequence:

```
git status
git add .
git push origin main
```

11. In my case, I noticed that the remote code in employee.js still does not have the last single apostrophe. In the local terminal, when I run a git status this is what I see:"



This means that we have modified models/employee.js, and those changes are currently staged. Git is holding them in a "preview" area, waiting for some kind of commit

12. First commit, then do a push:

```
git commit -m "Updated employee model"
git push origin main
```

Verify with a git status and check on the remote for that apostrophe.

At this point we have our local repo with three folders, index.js the two package files and a README.md file.

The other files are just supporting files for either the OS or Git or Node.

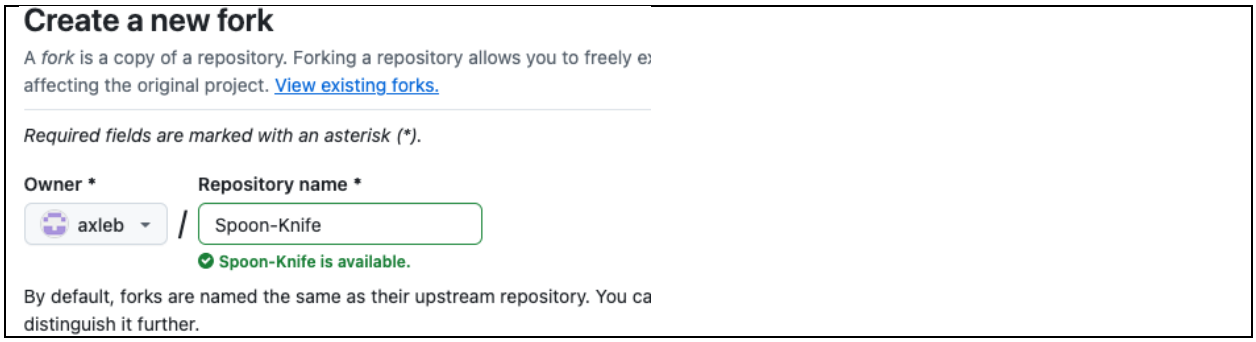
On the remote we have our main which is an exact copy of the local except for the node_module folder. We still have the feature-controller branch which will be deleted shortly.

Part 07 – Forking a Repo on the Remote

1. Cloning can access and copy code from a remote server to a personal computer, but Forking is making a copy of a project from someone else's GitHub account and storing it remotely on your own GitHub account. There is a repo created specifically for practicing. In your browser's GitHub UI, navigate to this repo:

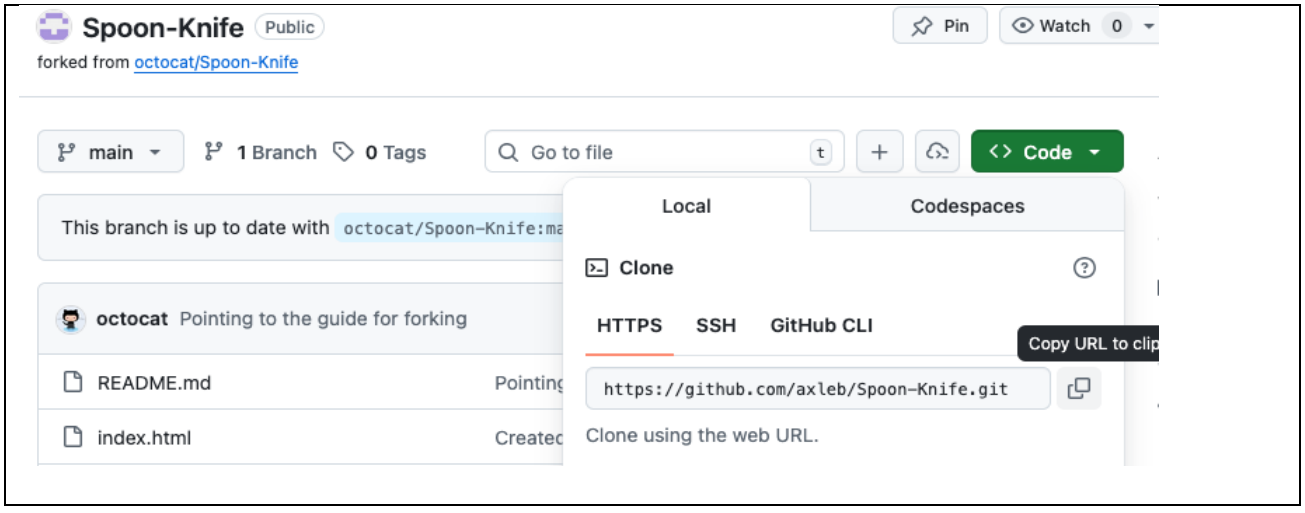
   https://github.com/octocat/Spoon-Knife

2. Click the `Fork` button in the top-right corner, then select `Create a new fork`. Your account will be the destination:



   Click the green `Create Fork` green button bottom right. It will now appear in the destination account as though that account owns this app.

3. Now we can bring this copy down to the local machine. Just like we did for the `git-node-d2` app. First get the remote address (green Code button):



4. Go into the Documents directory from VS Code. Now just clone like we did before:

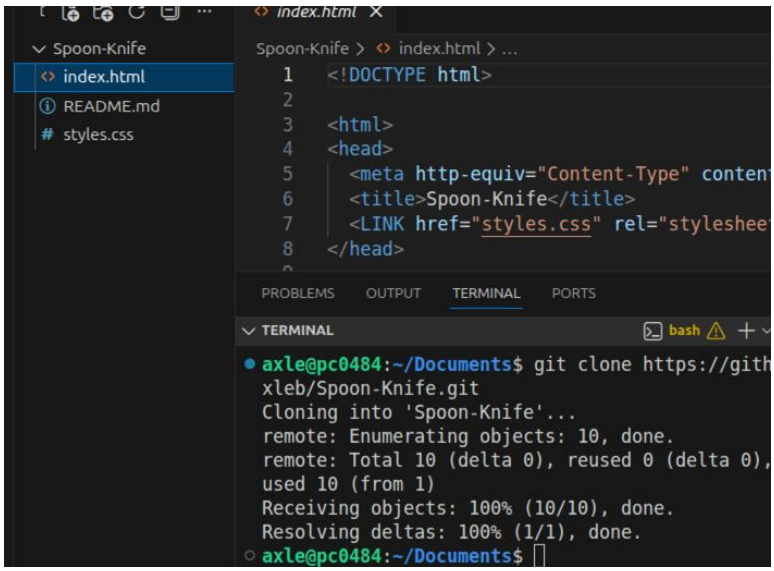
   git clone https://github.com/axleb/Spoon-Knife.git

   Now, just open the folder in VS Code. It is important to be on the Documents or any clean directory when you run this command.

5. This is the view from my Terminal window. This is why we installed Git globally, we can run this command from outside of a Git specific directory:

```
axle@pc0484:~$ cd Documents
axle@pc0484:~/Documents$ git clone https://github.com/axleb/Spoon-Knife.g
Cloning into 'Spoon-Knife'...
remote: Enumerating objects: 10, done.
remote: Total 10 (delta 0), reused 0 (delta 0), pack-reused 10 (from 1)
Receiving objects: 100% (10/10), done.
Resolving deltas: 100% (1/1), done.
axle@pc0484:~/Documents$ ls
git-node-d2  Spoon-Knife
axle@pc0484:~/Documents$
```

6. This is what my VS Code looks like now, as it is opened up to this new project:



7. Create a change and push back to GitHub. In VS Code change the title of index.html, for example:

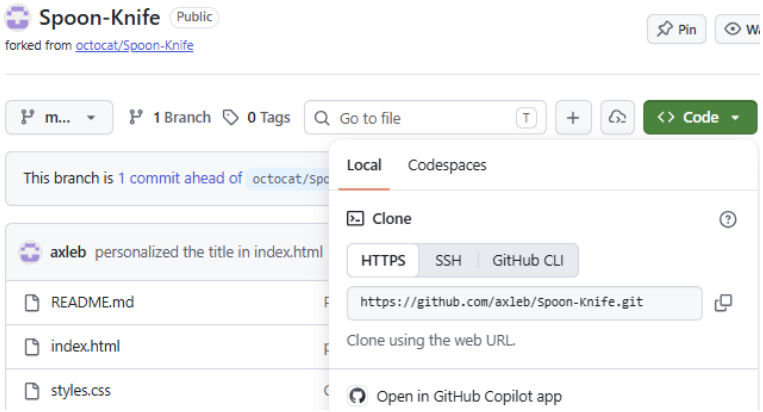
```
<head>
        <meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
        <title>Spoon-Knife-AxleB</title>
        <LINK href="styles.css" rel="stylesheet" type="text/css">
</head>
```

Of course you should use **your own** name here, NOT mine.

8. It is possible that the new directory on the local machine is not a Git project as yet, so make it Git by running the command:

```
git init
```

9. At this time, you have the copied repo under your account. So, before we can update that forked copy, we need to update our settings locally. So, under your account, click on the green Code button and copy the URL for cloning:



10. Once you have that address copied to your clipboard, go back to your local Spoon-Knife project and run this command:

```
git remote set-url origin https://github.com/axleb/Spoon-Knife.git
```

Remember, your account will be here NOT mine. Notice the git command is a bit different from what we used before.

11. If the directory has Git just do a git status to make sure that you can run Git commands. Since we made a change, lets do a git add:

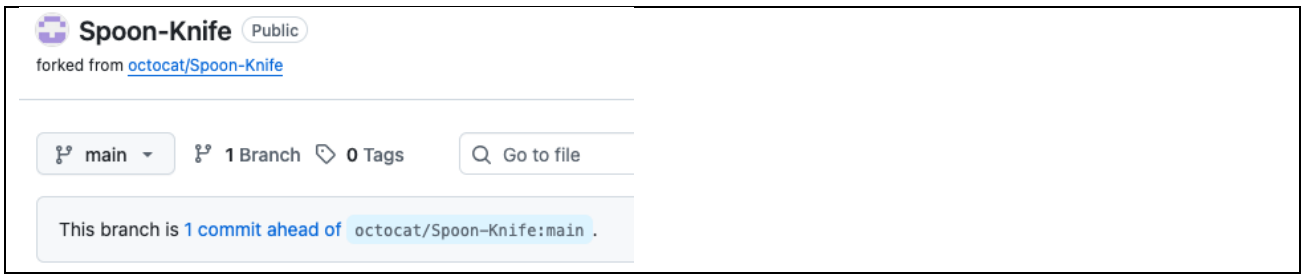
```
git add .
git commit -m “Changed the title”
```

12. After the `commit`, do a `push`

```
git push origin main
```

You will go through a few steps. Just click through until the push happens.

13. You will see a slightly different message. The branch is now ahead of the original repo because of the change to the index.html file:

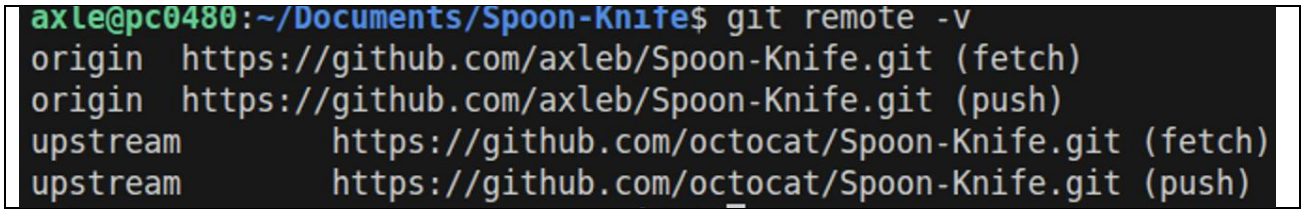


14. If you want to know when the original uploader updates their code, you need to link the local version to the original source:

```
git remote add upstream https://github.com/octocat/Spoon-Knife.git
```

This is called upstream

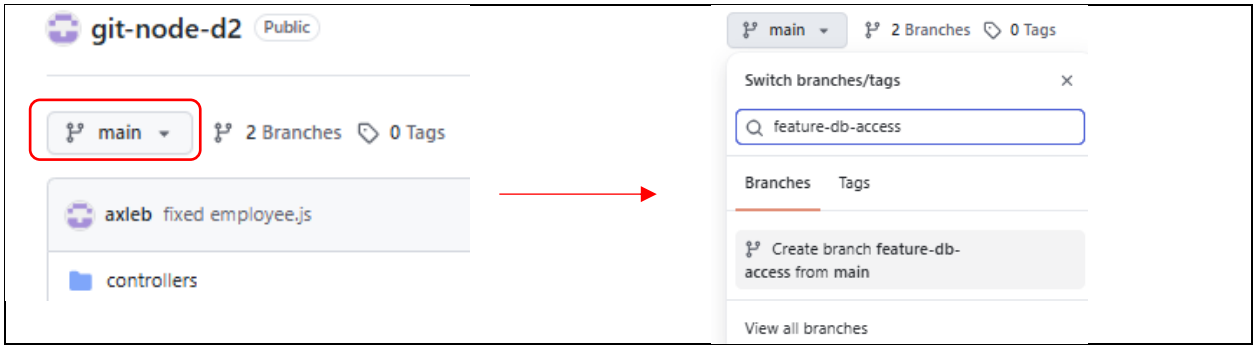
15. To check that this works, run the command `git remote -v`, you will see something similar to the image below:



Note, the Spoon-Knife Git project was created just for purposes like this. No one is monitoring the repo, so do not expect any kind of responses from the project.

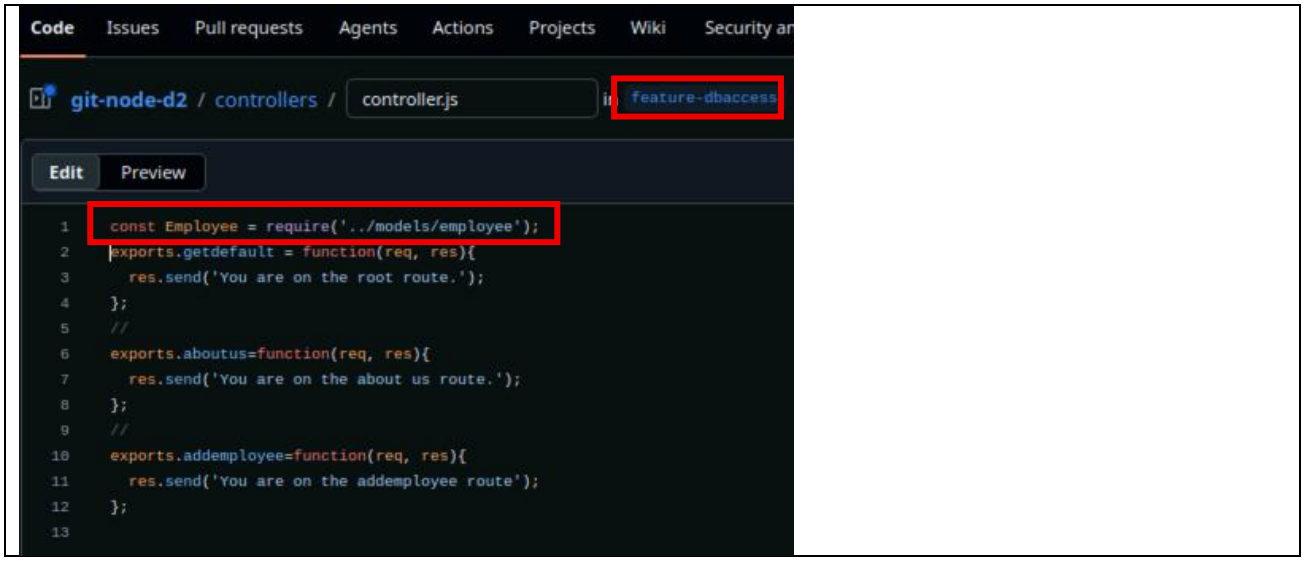
Part 08 – Branch on GitHub.com

1. Some tasks can be done on the remote server and then merged back to the local machine. For example, we can create a new branch of the `git-node-d2` using the web interface:



Here I have created a new branch called `feature-db-access`. Usually, you will be put on to the new branch once you click the `Create branch …` link. Remember on the web you will have to **manually** edit the file.

2. Still on the web interface, make sure you are on the new branch, then go to controller.js file, in the new branch, and add the line below:



You must be in **edit** mode to do this on the web. Remember to **Commit Changes**.

3. Now we need to add a function to this controller file to handle database documents (records). Add a new function to do this:

```
exports.getemployee= function(req, res){
  let empToFind = req.params.employeeName;
  Employee.find({})
  .then(
    employeeData => res.send(employeeData);
  )
  .catch((err)=>{
    res.send(err);
  })
};
```

4. This is what the function from #3 looks like on GitHub.com:



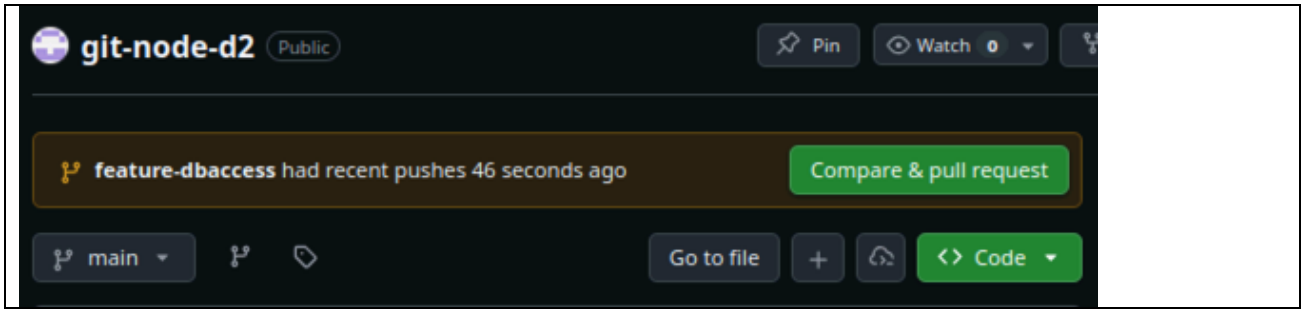
5. Just one more change in a different file. Once you commit the changes to the *controller* file open the routes.js file in the routes folder and add this line within the exported function:

```
router.get('/getemployees', controller.getemployees);
```



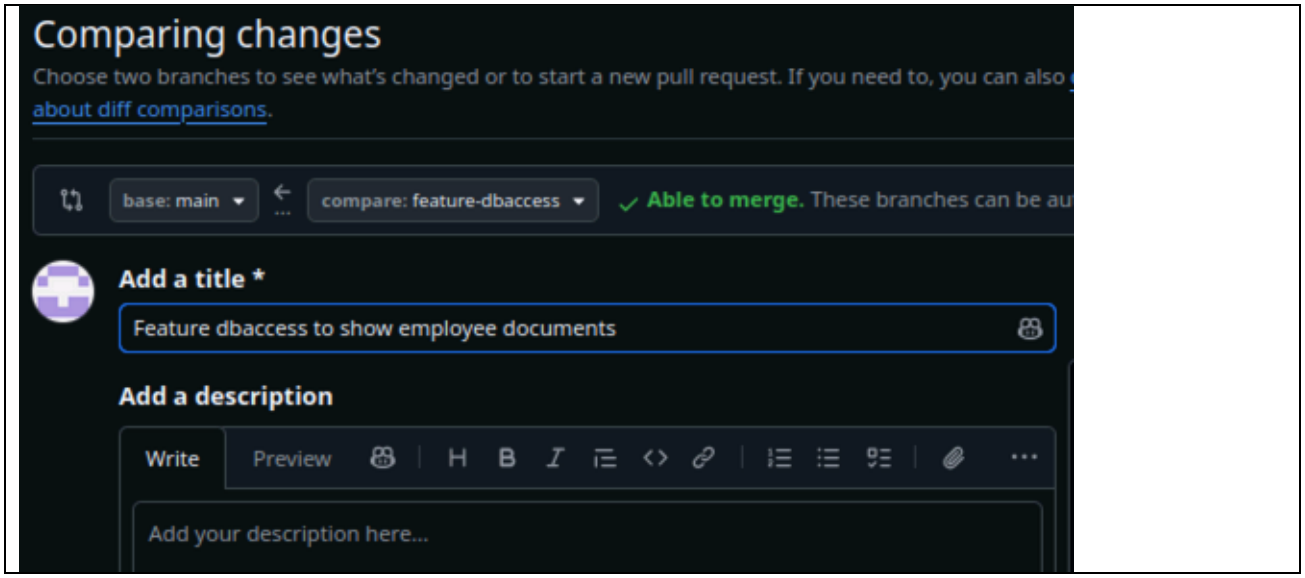
Remember you must be in **edit** mode and **commit** the changes once made.

6. We can now create a new Pull Request right there on the web. Switch to the `main` branch, a refresh should show a `Compare and pull` request button against a yellow background:

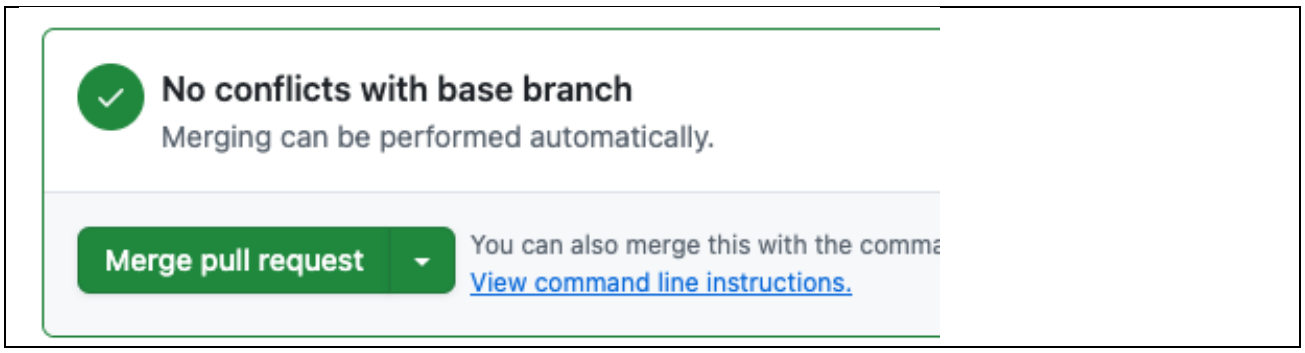


This is GitHub's way to tell you to open a Pull Request (PR) so you can review your changes, discuss them, and eventually merge them into your main branch

7. Clicking on that button will get you to the "Open a pull request" window. You can add a title and press the `Create pull request` green button on the bottom left:

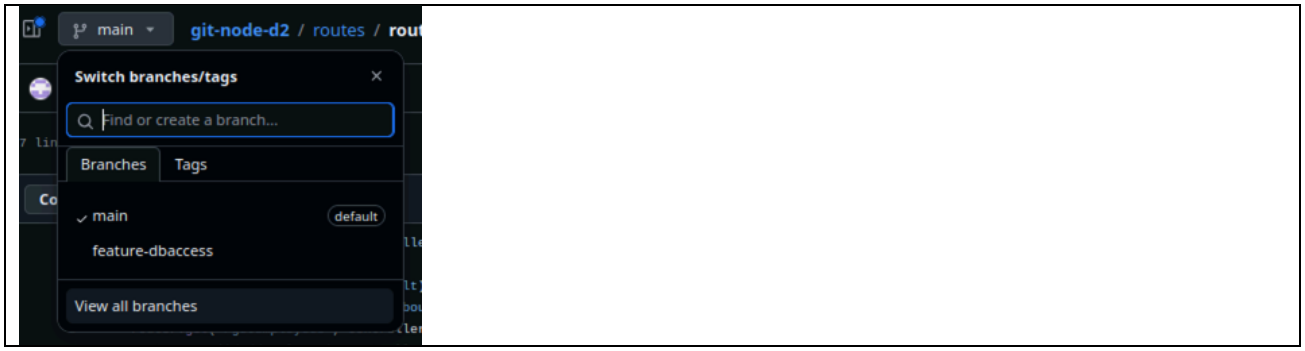


8. On the next interface, you will see the changes and you can then click on the `Merge pull request:`

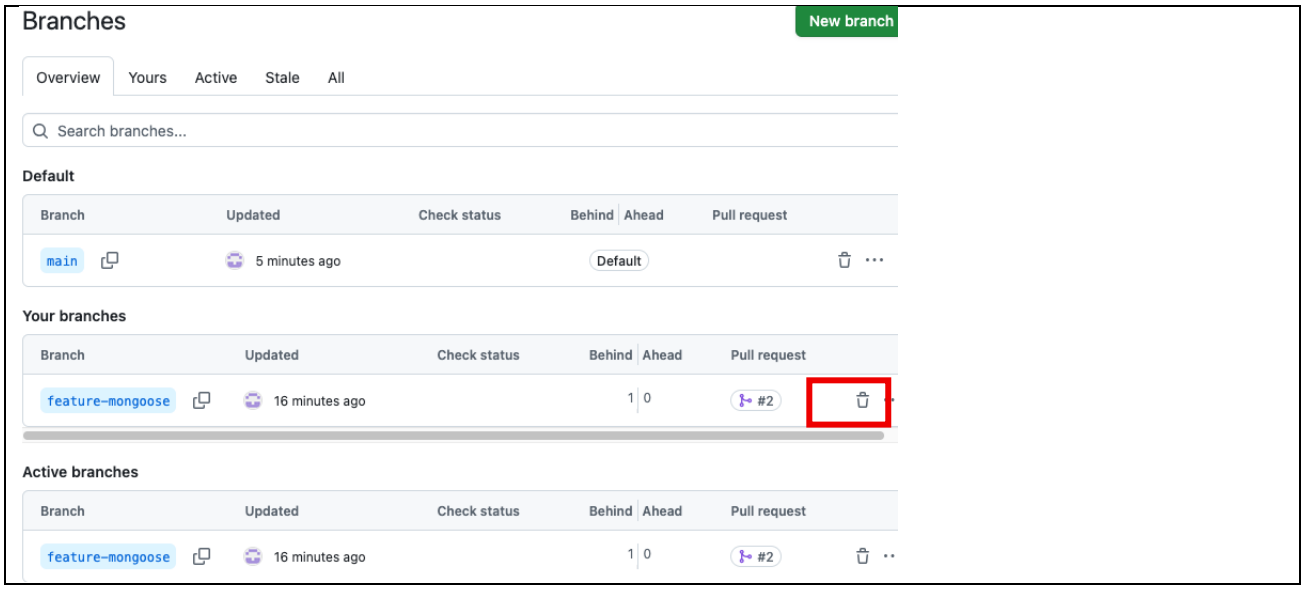


Confirm the merge.

9. Go back to the main branch and confirm that the controller.js fand routes.js files were updated. If you forgot to delete the `feature-dbaccess` branch after the merge, it is still possible. From the main branch click the downward facing arrow next to `main`, then click the `View all branches`:



10. This will get you to a place where all the branches exist and there is a garbage icon that you can use to remove any branch:



11. The last task if you ever did something like this is to synchronize with the local app. Follow the procedures listed in Part 04:

```
git fetch
git status
git diff main origin/main (optional)
git pull origin main
```

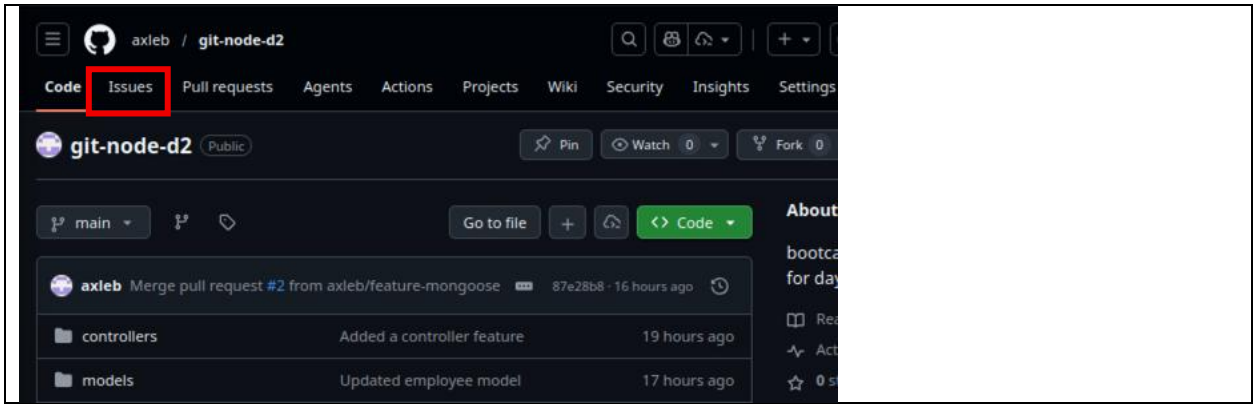
Note, if it's a new day or you just unzipped the local repo, it may not recognize remote edits. Run a `git fetch origin` one more time before continuing below.

Part 09 – Creating an Issue on GitHub

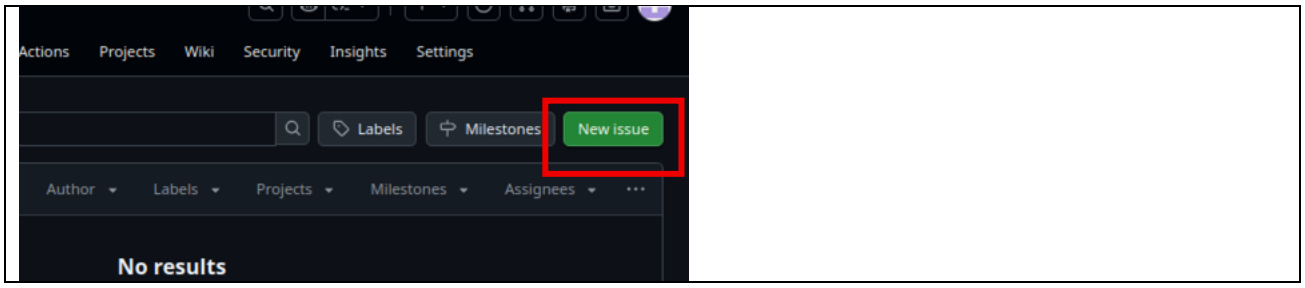
While a Pull Request is about proposing solutions, a GitHub Issue is about identifying problems, bugs or needed work. It serves as a digital "to-do" list or bug tracker for your repository.

With the current application, if you noticed, the /getemployees endpoint is not working properly. I will create an issue for this since I don't have the time to work on it now and maybe I can get some help. So let's document it so we don't forget the details.

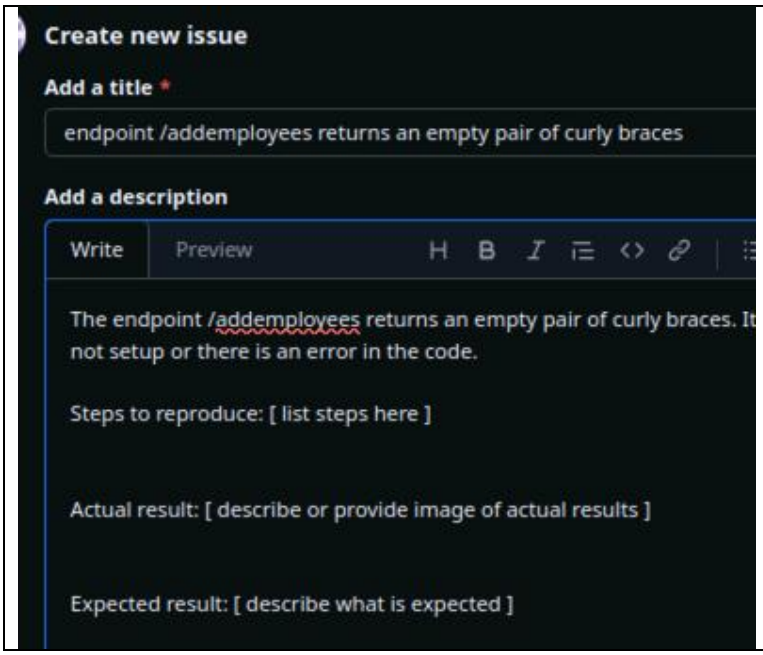
1. On your account in GitHub.com, the *issues* tab is right next to Code and to the left of Pull Requests. Click on that to get to the next screen:



2. Once on the new window, on the right hand side is the New Issue green button, click that to open the next window:



3. Once on the new window, enter a title and a description. In your description you should include the steps to reproduce the problem, the actual and expected results:

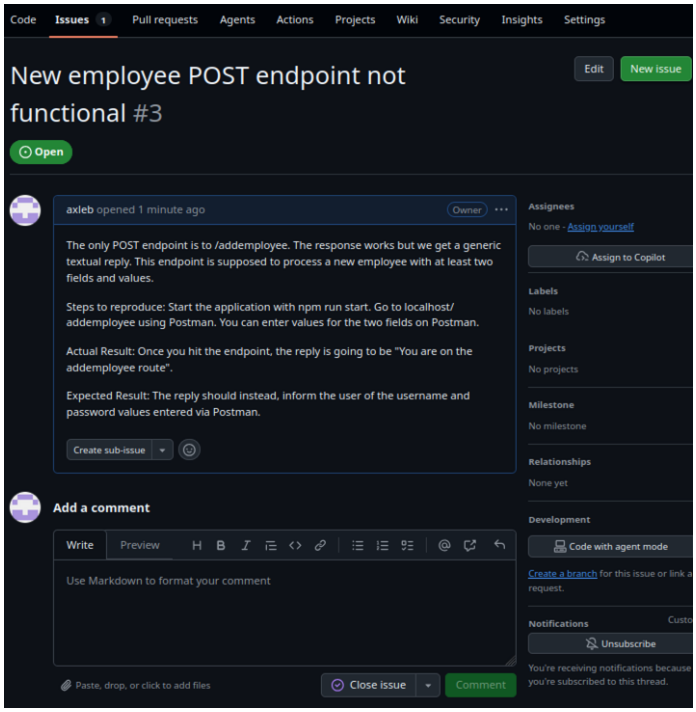


The endpoint /addemployees returns an empty pair of curly braces. It could be that a database was not setup or there is an error in the code.

Steps to reproduce: [list steps here]

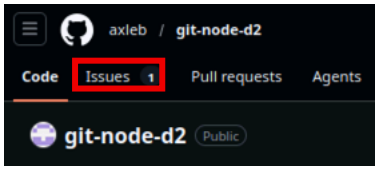
Actual result: [describe or provide image of actual results]

Expected result: [describe what is expected]:

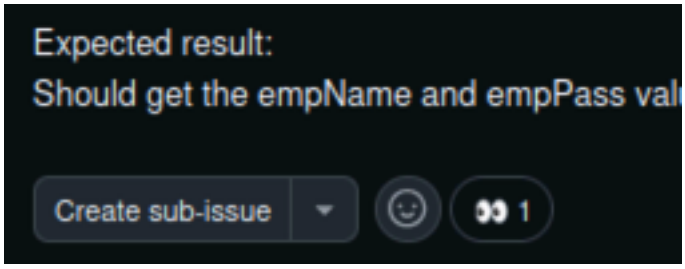


Once you have all the information documented, click on the green Create button on the bottom right.

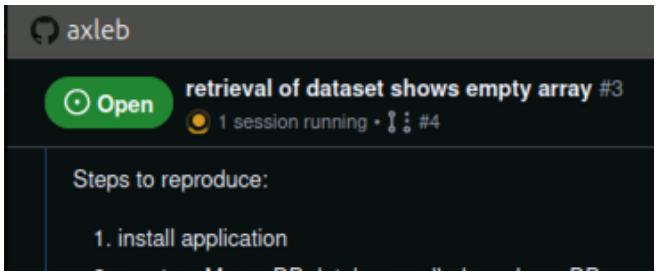
4. The issue is now a permanent part of your project history. Once it's created, it receives a number (e.g., #3). You can now refer to this number in your Git commits later to automatically link your code fix to the problem description.



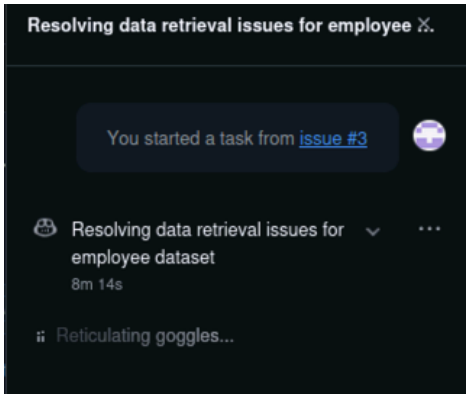
5. If you want to try the “Assign to an Agent” you must have a GitHub account. Once you assign the issue to the agent, check your email for updates. Once copilot has taken up the challenge, you will see an icon like this:



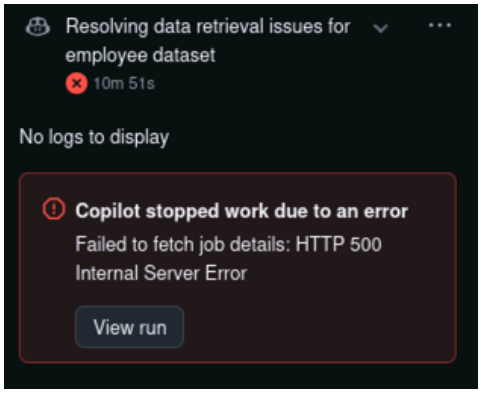
6. Also a session will start automatically a few seconds later:



7. You can also view the session to see what Copilot is doing



8. In my case the bug was not fixed:



9. It seems that Copilot, even though is working on it's own server, requires the local Angular server to be started. I started the server and after a few minutes, it completed a fix:

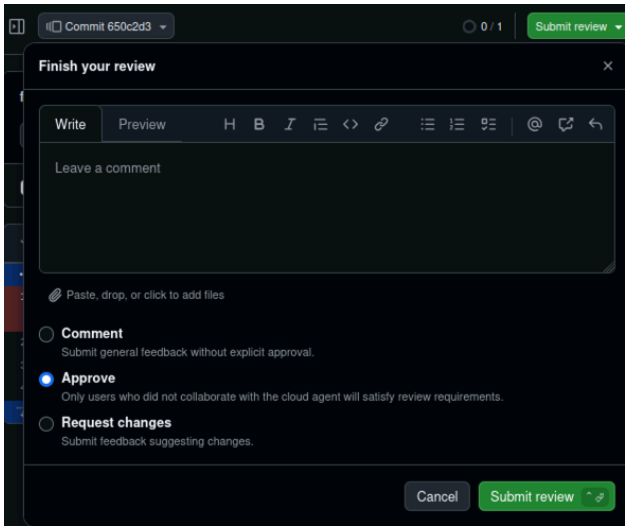


10. I opened the Review invitation and found this:



So in this case, Copilot, working on it's own found a bug and is asking me to approve the fix.

11. I approved and submitted the answer:



Appendix A – Tracking a Fresh Directory

Sometimes you may have a directory with an app in it that was never tracked by Git. If this happens you need to go through a few steps to make sure that Git is setup and ready to work with the folder.

1. Initialize Git with the command `git init`
2. Create a file named .gitignore in your root directory and add these lines:
   ```
   node_modules/
   .env
   ```
3. Usually if this is a fresh folder, just start tracking everything so `git add .`
4. Then `git commit -m "Initial commit: basic Node app"`

Appendix B – Fresh Token Generation

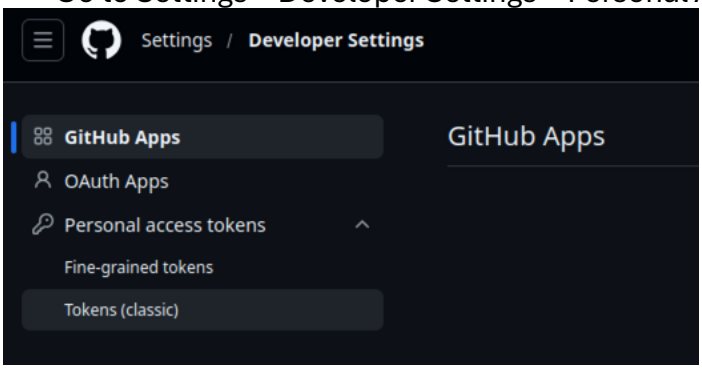
Sometimes you may have to request a token to complete a GitHub transaction. In my case I wanted to push a change made to a forked repository and I got this message in the terminal window:

```
axle@pc0484:~/Documents/Spoon-Knife$ git push origin m
ain
remote: Invalid username or token. Password authentica
tion is not supported for Git operations.
fatal: Authentication failed for 'https://github.com/a
xleb/Spoon-Knife.git/'
```

GitHub now requires a Personal Access Token (PAT) or an SSH Key to verify it's really you. The fastest way to get your push working right now is to generate a temporary token to use as your "password."

Generate a Token on GitHub.com:

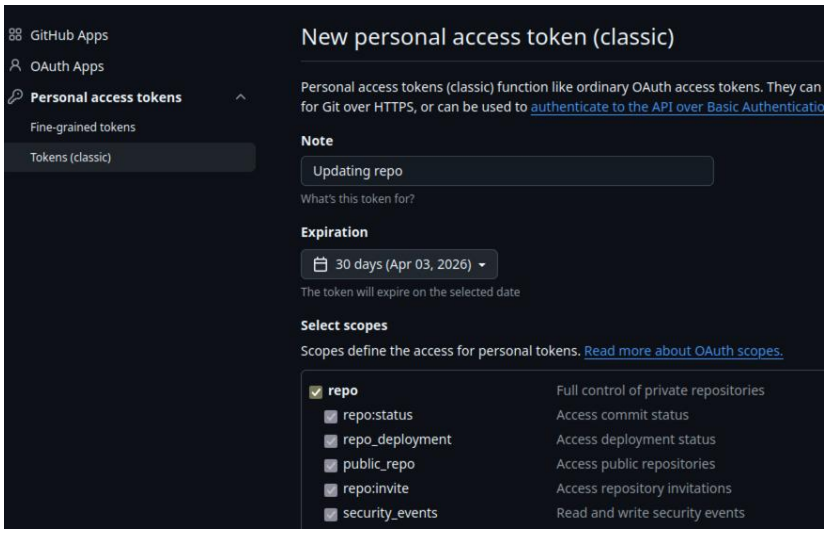
Go to Settings > Developer Settings > Personal Access Tokens > Tokens (classic).



Click Generate new token.

Give it a name (e.g., "git bootcamp") and select the repo scope (this allows you to push code).

Copy the token immediately. You won't see it again. You may get an email as well.



Make sure you check the `repo` checkbox then confirm everything at the bottom of the page.

Configure the remote URL:

git remote set-url origin https://<YOUR_TOKEN>@github.com/axleb/Spoon-Knife.git

it is possible that you may still get an authentication error like this:

```
axle@pc0484:~/Documents/Spoon-Knife$ git push origin main
remote: Permission to axleb/Spoon-Knife.git denied to axleb.
fatal: unable to access 'https://github.com/axleb/Spoon-Knife.gi
t/': The requested URL returned error: 403
```

We need to force the push command to use this particular account, meaning your account:
git remote set-url origin https://**<your_github_username>**:<your_token>@github.com/axleb/Spoon-Knife.git

Appendix C – Show .git Folder in VS Code

If you want to show the .git folder in VS Code, this is one of the ways to do it. File -> Preferences -> Settings. Once there type `files.exclude` and adjust as needed: